

AI System — Prompt Engineering & State Machine Logic

Overview

BioSync uses Groq's Llama 3.1 (8B) as its inference engine. The model is not used as a general chatbot — it operates as a **state-aware, memory-informed intervention system** that behaves differently in each of the three cognitive states.

System Prompt: The BioSync Sentinel

```
You are the BioSync Sentinel – an edge-resident cognitive intelligence system.
Your purpose: maintain the user's cognitive performance by providing precise,
state-appropriate interventions based on real-time biosignal data.
You receive:
- stress_score: number (0-100, current cognitive load)
- state: "CALM" | "LOAD" | "CRITICAL"
- failure_memory: string[] (list of interventions that failed this session)
- user_input: string (optional – transcribed speech or typed text, may be Hinglish)
BEHAVIORAL RULES BY STATE:
■ CALM (score < 40):
- The user has cognitive bandwidth. Use it.
- Provide ambitious, forward-looking guidance.
- Suggest deep work, planning, creative tasks.
- Tone: strategic, confident, minimal.
- Max response: 2 sentences.
■ LOAD (score 40-75):
- The user is under pressure. Reduce friction.
- Convert any unstructured input into structured bullet points.
- Eliminate all preamble. Start with the action.
- Never ask clarifying questions.
- Tone: precise, operational.
- Max response: 3 bullet points.
■ CRITICAL (score > 75):
- The user needs immediate de-escalation.
- Check failure_memory. NEVER suggest an intervention already in that list.
- Give ONE single directive. Not a list. Not options. One thing.
- Tone: calm, direct, clinical.
- Max response: 1 sentence.
- If failure_memory contains all standard interventions, escalate:
suggest the user notify a teammate and step away for 10 minutes.
LANGUAGE:
- If user_input contains Hinglish (mixed Hindi-English), respond in the same
register – don't force formal English.
- Example: "Yaar, bohot zyada kaam hai" → respond with operational Hindi-English mix.
- Never use generic empathy phrases ("I understand how you feel", "That sounds hard").
- BioSync does not empathize. BioSync optimizes.
FAILURE MEMORY ENFORCEMENT:
- failure_memory is a blacklist for this session.
- If "Breathing Exercise" is in failure_memory, do not mention breathing.
```

```
- If "Task Decomposition" is in failure_memory, do not suggest breaking tasks down.  
- If you have no non-blacklisted interventions, trigger escalation protocol.
```

Request Schema

```
interface BioSyncAIRequest {  
  stress_score: number; // 0-100  
  state: 'CALM' | 'LOAD' | 'CRITICAL';  
  failure_memory: string[]; // Intervention names that failed  
  user_input?: string; // Optional: transcribed Hinglish/English  
  session_duration_minutes: number; // Context for fatigue assessment  
}
```

Response Schema

```
interface BioSyncAIResponse {  
  intervention: string; // The response to display  
  intervention_type: InterventionType; // For logging to failure memory  
  confidence: number; // 0-1, model confidence  
  should_escalate: boolean; // True if all interventions exhausted  
}  
  
type InterventionType =  
  | 'TASK_STRIPPING'  
  | 'BREATHING_EXERCISE'  
  | 'MICRO_BREAK'  
  | 'ENVIRONMENTAL_RESET'  
  | 'DEEP_WORK_PROMPT'  
  | 'ESCALATE_HUMAN'  
  | 'FOCUS_REFRAME';
```

Groq Client Implementation

```
// ai/groqClient.ts  
import Groq from 'groq-sdk';  
  
const groq = new Groq({  
  apiKey: process.env.NEXT_PUBLIC_GROQ_API_KEY,  
  dangerouslyAllowBrowser: true,  
});  
  
export async function getBioSyncIntervention(  
  request: BioSyncAIRequest  
): Promise<BioSyncAIResponse> {  
  const userMessage = buildUserMessage(request);  
  const completion = await groq.chat.completions.create({  
    model: 'llama-3.1-8b-instant',  
    messages: [  
      { role: 'system', content: BIOSYNC_SYSTEM_PROMPT },  
      { role: 'user', content: userMessage },  
    ],  
    max_tokens: 120, // Keep responses tight and fast  
    temperature: 0.3, // Low temp = consistent, clinical tone  
    stream: false,  
  });  
  
  const responseText = completion.choices[0].message.content ?? '';  
  return parseAIResponse(responseText, request.state);  
}
```

```
function buildUserMessage(request: BioSyncAIRequest): string {
return JSON.stringify({
stress_score: request.stress_score,
state: request.state,
failure_memory: request.failure_memory,
user_input: request.user_input ?? 'none',
session_duration_minutes: request.session_duration_minutes,
});
}
```

Intervention Priority Tree

When the AI response includes an `intervention_type`, the system also has a deterministic fallback tree for offline mode or when Groq latency exceeds 1 second:

```
// ai/interventionTree.ts
const INTERVENTION_PRIORITY: Record<string, InterventionType[]> = {
CRITICAL: [
'TASK_STRIPPING', // Priority 1
'BREATHING_EXERCISE', // Priority 2
'ENVIRONMENTAL_RESET', // Priority 3
'MICRO_BREAK', // Priority 4
'ESCALATE_HUMAN', // Priority 5 (last resort)
],
LOAD: [
'FOCUS_REFRAME',
'TASK_STRIPPING',
'MICRO_BREAK',
],
CALM: [
'DEEP_WORK_PROMPT',
'FOCUS_REFRAME',
],
};

export function getOfflineIntervention(
state: 'CALM' | 'LOAD' | 'CRITICAL',
failureMemory: string[]
): InterventionType {
const priority = INTERVENTION_PRIORITY[state];
// Find first intervention not in failure memory
const available = priority.find(
intervention => !failureMemory.includes(intervention)
);
return available ?? 'ESCALATE_HUMAN';
}
```

State Machine Logic

```
// utils/stressEngine.ts
interface StateTransition {
from: CognitiveState;
to: CognitiveState;
timestamp: number;
score: number;
}
```

```

class CognitiveStateMachine {
private state: CognitiveState = 'CALM';
private scoreBuffer: number[] = [];
private escalationCounter = 0;
private deescalationCounter = 0;
private readonly ESCALATION_HOLD = 3; // consecutive windows to escalate
private readonly DEESCALATION_HOLD = 6; // consecutive windows to de-escalate
update(score: number): CognitiveState {
this.scoreBuffer.push(score);
if (this.scoreBuffer.length > 10) this.scoreBuffer.shift();
const targetState = this.scoreToState(score);
if (targetState > this.stateRank(this.state)) {
// Potential escalation
this.escalationCounter++;
this.deescalationCounter = 0;
if (this.escalationCounter >= this.ESCALATION_HOLD) {
this.state = targetState;
this.escalationCounter = 0;
}
} else if (targetState < this.stateRank(this.state)) {
// Potential de-escalation
this.deescalationCounter++;
this.escalationCounter = 0;
if (this.deescalationCounter >= this.DEESCALATION_HOLD) {
this.state = targetState;
this.deescalationCounter = 0;
}
} else {
this.escalationCounter = 0;
this.deescalationCounter = 0;
}
return this.state;
}
private scoreToState(score: number): CognitiveState {
if (score < 40) return 'CALM';
if (score < 75) return 'LOAD';
return 'CRITICAL';
}
private stateRank(state: CognitiveState): number {
return { CALM: 0, LOAD: 1, CRITICAL: 2 }[state];
}
}

```

Why asymmetric hold times?

Escalating quickly (3 windows $\approx$ 1.5s) means the system responds to real stress events promptly. De-escalating slowly (6 windows $\approx$ 3s) prevents the UI from flickering back to green the moment someone takes a breath — cognitive stress doesn't disappear in 1.5 seconds.

Hinglish NLP — Intent Classification

```

// utils/hinglishParser.ts
interface IntentResult {
intent: IntentClass;
score: number; // 0-1
language: 'hindi' | 'english' | 'hinglish';
keywords: string[];
}

```

```

}
type IntentClass =
| 'URGENT_DEADLINE'
| 'HIGH_STRESS'
| 'OVERWHELMED'
| 'FOCUS_REQUEST'
| 'NEUTRAL'
| 'POSITIVE';

const HINGLISH_PATTERNS: Array<{pattern: RegExp; intent: IntentClass; weight: number}> = [
// High urgency patterns
{ pattern: /deadline|kal\s*tak|aaj\s*tak|urgent|abhi/i, intent: 'URGENT_DEADLINE', weight: 0.9 },
{ pattern: /tension|stress|pareshaan|worried/i, intent: 'HIGH_STRESS', weight: 0.8 },
{ pattern: /bahut.*kaam|zyada.*kaam|overwhelm|cooked/i, intent: 'OVERWHELMED', weight: 0.85 },
{ pattern: /bhai.*yaar|arre\s*yaar|dude/i, intent: 'HIGH_STRESS', weight: 0.6 },
// Focus requests
{ pattern: /focus|concentrate|dhyaan|kaam\s*karna/i, intent: 'FOCUS_REQUEST', weight: 0.7 },
// Positive signals
{ pattern: /theek\s*hai|all\s*good|sab\s*sahi|calm/i, intent: 'POSITIVE', weight: 0.4 },
];

export function parseHinglishIntent(text: string): IntentResult {
const matches = HINGLISH_PATTERNS
.filter(p => p.pattern.test(text))
.sort((a, b) => b.weight - a.weight);
if (matches.length === 0) {
return { intent: 'NEUTRAL', score: 0.5, language: 'english', keywords: [] };
}
const topMatch = matches[0];
const keywords = text.match(topMatch.pattern) ?? [];
return {
intent: topMatch.intent,
score: topMatch.weight,
language: detectLanguage(text),
keywords: keywords.map(k => k.toLowerCase()),
};
}

function detectLanguage(text: string): 'hindi' | 'english' | 'hinglish' {
const hindiWords = (text.match(/\b(hai|kaam|yaar|bhai|bahut|zyada|aaj|kal)\b/gi) ?? []).length;
const totalWords = text.split(/\s+/).length;
const hindiRatio = hindiWords / totalWords;
if (hindiRatio > 0.6) return 'hindi';
if (hindiRatio > 0.2) return 'hinglish';
return 'english';
}

```